PROJECT REVIEW & IMPROVEMENT ROADMAP

Kai Portfolio

From a collection of experiments
to a focused, dependable showcase.



16 review briefs. One clear reference.

15 games and prototypes + the portfolio hub.
Source-backed findings, real browser observations,
priorities, acceptance checks and a staged roadmap.

PREPARED FOR KAI & TIM / 28 SEPTEMBER 2026

Repository: hasturofyellow/kai-portfolio | Verified main: 75a9e07

The strongest next version

The portfolio already demonstrates range: real-time games, incremental economies, branching stories, multiplayer experiments, device sensors and two immersive venues. The biggest opportunity is to make that range **reliably playable, easy to discover and clearly explained**.

Recommendation Repair the central loops first. Then curate three featured projects and apply a small shared quality standard to the rest. Adding more games now would increase the maintenance burden before fixing the visitor experience.

Start with these release blockers

Area	Why it matters	Effort
Chain Reaction	The inline script does not parse; the board never appears.	S
Snake arena	Local simulation is missing; server steering is reset before use; food snapshots are not rendered.	M
God game	Character loaders request a directory that is absent from the repository.	S
Cheeseburger Empire	Bulk sale produces an unlimited net gain after reaching 500 burgers.	S-M
Portfolio links	Idle.html disagrees with tracked idle.html on case-sensitive hosts.	S

Build a memorable featured lineup

1. Kai's Cinema: the flagship venue

Keep its atmosphere and existing animation systems. Add a direct project browser and optional guided commentary so the work is reachable even when movement controls fail.

2. Blastblock or Age of War: the polished game

Pick one for the first complete quality pass. Blastblock offers a smaller scope; Age of War offers deeper systems but more balancing work. Make the choice after a clean playtest.

3. NULL: the distinctive experiment

Its sensor-based concept broadens the portfolio. Pair it with real-device validation, understandable capability failures and accessible alternatives.

These are editorial recommendations, not measured popularity rankings. No analytics, player survey or performance benchmark was available.

Guide to this reference

Use this report as a backlog and review checklist. Project briefs combine existing strengths, firsthand observations, four prioritized actions and concrete acceptance checks. Each evidence link opens the exact reviewed GitHub revision.

Section	Pages
Executive recommendation	2
Complete inventory	4
Hands-on playtest log	5-6
16 project briefs	7-22
Shared portfolio and engineering work	23-24
Release roadmap and first ten work items	25-26
Definition of ready	27
Browser evidence	28-30
Evidence, provenance and limits	31

Priority and effort legend

P0 - the central experience cannot function correctly, or its core economy is invalid.
P1 - reliability, navigation, accessibility or interaction work needed before wider sharing.
P2 - polish, differentiation or maintenance that follows a dependable baseline.

S - up to 1 focused developer day. **M** - 2-4 days. **L** - 5-10 days. These are planning estimates for the stated scope, not quotes or deadlines. Acceptance work, device access and unexpected dependencies can change them. Do not sum overlapping recommendations as separate projects.

Evidence labels Observed means reproduced through the browser. Partial means only the stated interaction was attempted. Source findings describe implementation facts; risks describe consequences requiring further validation. Proposed targets are not measured results.

Snapshot: local tracked source matched GitHub main at **75a9e072f14df546e5fcae9fe364ac7ef0578263**, committed 18 May 2026. Verified on 28 September 2026. No project source was changed during the review.

Complete project inventory

All tracked HTML application entry points were included: one hub plus fifteen games/prototypes. Asset packs, screenshots and planning documents are supporting material, not additional playable projects.

Project	Entry point	Hub	Page
Portfolio Hub	index.html	Home	7
Kai's Cinema	3D Game Engine/index.html	Unlisted	8
Theater of Projects	legoTest.html	Unlisted	9
Snake Game	snakeGame.html	Listed	10
Blastblock	blastblock.html	Listed	11
Blastblock 9x9	blockblastcopy.html	Unlisted	12
Doodle Jump	doodleJump.html	Listed	13
Age of War	ageOfWar.html	Listed	14
Stick Soccer Showdown	stick/index.html	Listed	15
Cheeseburger Empire	idle.html	Listed*	16
Cosmic Cookie Clicker	cookieClicker.html	Unlisted	17
Sharknado Adventure	sharkNado.html	Listed	18
Chain Reaction / Color Wars	colorWars.html	Unlisted	19
NULL / Sensor Puzzle	sensor-puzzle/index.html	Unlisted	20
2D Engine / Snake demo	Game Engine/index.html	Unlisted	21
God game	God game/index.html	Unlisted	22

* Cheeseburger Empire is listed as Idle.html; the tracked file is idle.html. This works on a case-insensitive Windows filesystem but is a deployment risk on a case-sensitive host.

Curation decision Use one canonical Blastblock engine with named 8x8 and 9x9 modes, or archive the older variant. Choose one primary immersive portfolio and explain the other as an earlier experiment or an alternative interaction study.

Hands-on playtest log / 1

Test environment: the reviewed commit served from 127.0.0.1:8765 in the visible Codex in-app Chromium browser on Windows. Actions used real UI controls; no game state was injected. These are short exploratory sessions, not full compatibility or balance certification.

Portfolio Hub Observed

Search for snake and Strategy filtering each returned the correct card. An unmatched query left a blank grid. Card titles were hidden in the unhovered view; Tab navigation did not expose the cards as links.

Kai's Cinema Partial / environment limit

The neon exterior and animated characters rendered. W and Space inputs produced movement/jump changes, but pointer capture failed with a Chromium error and the start overlay remained. Interior traversal and ticket interactions were not validated.

Theater of Projects Partial

Drag-to-look changed the view; help and reset responded, and the popcorn total became 26. E from the spawn area produced no visible project launch. The scene felt very dark. Full portal traversal was not completed.

Snake Game Local + online checked

Local Start displayed a static arena; mouse/Space produced no progression. A separate local WebSocket session joined successfully and moved the player, but steering attempts did not visibly redirect it and no food appeared. Two-client synchronization and reconnect were not tested.

Blastblock Partial / input limit

The board, pieces and Reset loaded. Two browser drags did not place a piece. The input tool may not reproduce native HTML drag-and-drop, so this is not a confirmed user-facing drag failure. The piece tray was partly below the visible fold.

Blastblock 9x9 Partial / input limit

The board and hand rendered; a drag did not produce placement. Empty parts of the plus/T silhouettes appeared as colored cells, matching the CSS conflict. A complete scoring run was not validated.

Doodle Jump Short interaction

The character bounced and responded to direction taps; a projectile was visible after keyboard input. The score stayed at zero in this short run. No instructions were visible, and a complete climb or post-loss run was not validated.

Age of War Played opening battle

Started Easy, purchased Clubman and Slingshot units, then bought a Volcano Turret. Units entered combat. Upgrade Base gave no explanation when unaffordable. The letter-based units were difficult to distinguish by role. Match completion was not tested.

Hands-on playtest log / 2

Test environment: the reviewed commit served from 127.0.0.1:8765 in the visible Codex in-app Chromium browser on Windows. Actions used real UI controls; no game state was injected. These are short exploratory sessions, not full compatibility or balance certification.

Stick Soccer Showdown Short interaction

Started a Normal match and tried direction, Space and X inputs while the ball and opponent moved. No goal was scored in the session. The compact controls text disappeared during play; goal/replay edge cases remain source-based findings.

Cheeseburger Empire Played + reload check

Grilled 50 burgers, bought a Dorm Shop and observed 1 burger per second with the next shop priced at 75. Reload reset ownership and production to zero. The unlimited bulk-sale issue was established from source, not reached in this run.

Cosmic Cookie Clicker Played + save check

Clicked 20 times, bought one Cursor for 15, saved and reloaded. Ownership persisted. The production display still read 0 per second despite the Cursor's fractional output. Early purchases and visual feedback felt responsive.

Sharknado Adventure Completed one route

Chose emergency kit > search house > follow map > solve puzzle > search for survivors. Reached Hopeful Unity at 75 points after five choices; Restart returned to 50. Some image transitions lagged or appeared empty; all remote assets were not audited.

Chain Reaction / Color Wars Observed startup blocker

The controls appeared but no board loaded. Clicking New Game did nothing. Browser logs reported: Identifier p has already been declared. No legal move could be attempted.

NULL / Sensor Puzzle Desktop only

Allowed motion onboarding, opened Be Still and later saw 1/4 solved without physical device interaction. Opened the shake puzzle and used Back. This validates desktop navigation only; sensor samples, phone permissions and actual physical puzzles remain unverified.

2D Engine / Snake Played restart/turn

The grid demo started immediately and reached Game Over. Space restarted it, and a Down input changed the movement direction. The controls are not explained before play. Food collection and a full-board win were not tested.

God game Observed asset blocker

The camera and tree/hut scenery rendered and responded to dragging. Character loads repeatedly failed with assets/dude/dude.babylon not found. No units could be selected. The grass texture displayed a visible watermark; replace it with a documented, suitable asset.

Portfolio Hub

index.html | Review brief 01 of 16

What is worth keeping. The lightweight homepage has a clear personal introduction, visual thumbnails, live search and category filtering. It starts without a build step. Keep the direct browsing experience as the dependable front door, with immersive worlds offered as an optional way to explore the same work.

Browser: Observed Search for snake and Strategy filtering each returned the correct card. An unmatched query left a blank grid. Card titles were hidden in the unhovered view; Tab navigation did not expose the cards as links.

P1 / S 1. Make the catalog complete and accurate

Only seven of fifteen game/prototype entry points appear in the registry. Cheeseburger Empire points to Idle.html although the tracked file is idle.html. Create one project manifest for all portfolio surfaces; classify ready projects, experiments and archived variants. Fix exact-case URLs and check every destination.

Done when: Every public project has an intentional status and working link; experimental items are clearly labeled and discoverable.

index.html:257 | index.html:298

P1 / M 2. Make cards usable without hovering

Project titles and descriptions start hidden and appear on hover; cards are clickable divs without link semantics. Use real anchor cards with visible titles, focus styles and concise device/control labels. Provide separate Play and Case Study links where useful.

Done when: Keyboard and touch visitors can identify and open every card; content remains readable without hover or animation.

index.html:127 | index.html:331 | index.html:335

P1 / S 3. Explain search and navigation state

Search and filtering work, but no-results searches simply empty the grid. Add persistent labels, a result count, an empty-state explanation and Clear Filters. Give each game a consistent return-to-portfolio control so visiting a project does not strand the visitor.

Done when: An unmatched query explains what happened; clearing filters restores the catalog; every game provides a reliable route home.

index.html:240 | index.html:328 | index.html:360

P2 / M 4. Demonstrate the work behind each game

The introduction uses age-specific copy and the footer is fixed to 2025. Verify the biography with Kai and use durable wording. Add role, technical challenge, implementation decision, screenshot, source link and what was learned for featured projects; update metadata and page titles.

Done when: A visitor can understand Kai's contribution and one specific engineering achievement for each featured project within a minute.

index.html:217 | index.html:252

Distinctive next step. Feature three contrasting strengths: an immersive venue, a refined arcade game and an experimental interaction. Keep the complete catalog underneath; give experiments context without presenting unfinished games as release-ready.

Kai's Cinema - 3D Game Engine

3D Game Engine/index.html | Review brief 02 of 16

What is worth keeping. The most developed immersive venue has modular locations, animated characters, dialogue, seating, collectible currency, theater screens, lava platforming and arcade minigames. Coyote time and jump buffering improve movement forgiveness. Bloom is already implemented. These working systems provide a strong foundation for a flagship portfolio experience.

Browser: Partial / environment limit The neon exterior and animated characters rendered. W and Space inputs produced movement/jump changes, but pointer capture failed with a Chromium error and the start overlay remained. Interior traversal and ticket interactions were not validated.

P1 / M 1. Offer an accessible visitor route

Camera input requires pointer lock, movement is keyboard-only, and theater entry consumes tickets. Add drag-to-look, visible controls and a free project browser; keep currency as optional exploration. Add touch actions through the same input abstraction so visitors can reach the work directly.

Done when: Every project opens without earning currency; desktop works without pointer lock, and touch visitors can navigate the project browser.

3D Game Engine/src/input.js:45 | 3D Game Engine/src/input.js:79 | 3D Game Engine/src/game/theaterGame.js:1239

P1 / M 2. Make scene resource ownership explicit

FBX callbacks add models without checking whether their location still exists, while disposal handles only top-level geometry and materials. This creates a risk of stale models and retained GPU resources. Guard asynchronous loads with scene generations and recursively dispose owned geometry, materials and textures.

Done when: Under throttled loading and twenty scene round trips, old models never appear and resource counts stabilize after warm-up.

3D Game Engine/src/game/locations/exterior.js:54 | 3D Game Engine/src/game/locations/exterior.js:891 | 3D Game Engine/src/game/locations/lobby.js:1189

P2 / M 3. Preserve optional exploration progress

Wallet, tickets and collected bills initialize in memory and have no persistence layer. Introduce a versioned save object with validation, recovery and explicit reset. Keep presentation settings alongside it, and save after transactions instead of tying storage writes to the render loop.

Done when: Reload retains valid progress; invalid saves recover gracefully; repeated transactions cannot duplicate rewards; reset requires an explicit action.

3D Game Engine/src/game/theaterGame.js:38 | 3D Game Engine/src/game/theaterGame.js:429 | 3D Game Engine/src/game/theaterGame.js:567

P2 / M 4. Align architecture and roadmap with implementation

Documentation incorrectly describes bloom and seated animation as absent, while the main theater remains a TODO. Update the feature inventory, give unfinished destinations clear labels, and extract dialogue, economy and arcade code from the large controller before adding more rooms or effects.

Done when: Roadmap matches active code; every signed destination has an intentional result; transaction and transition behavior survives module extraction.

3D Game Engine/src/game/theaterGame.js:86 | 3D Game Engine/src/game/theaterGame.js:892 | 3D Game Engine/src/game/locations/lobby.js:694 | 3D Game Engine/movieTheaterProject.md:202

Distinctive next step. Build a director's-commentary tour: each screen presents the playable project, its hardest engineering problem and a short making-of story. Optional exploration reveals extras while a guided route respects visitors' time.

Theater of Projects - Lego-style Venue

legoTest.html | Review brief 03 of 16

What is worth keeping. This alternative theater already includes drag-to-look, buffered double jumping, procedural sound, collectible popcorn, a VIP lounge, an animated usher and easter eggs. Project posters function as destinations. The varied feedback systems give the venue a distinctive playful identity, with substantial existing content worth preserving.

Browser: Partial Drag-to-look changed the view; help and reset responded, and the popcorn total became 26. E from the spawn area produced no visible project launch. The scene felt very dark. Full portal traversal was not completed.

P0 / S 1. Correct the case-sensitive project link

The Cheeseburger Empire poster links to Idle.html, but the committed filename is idle.html. Correct the spelling and add a case-sensitive local-link check for project data. Windows can mask this mismatch, so validate exact paths against the deployment artifact rather than relying on local navigation.

Done when: Every poster URL matches a committed destination exactly, and all destinations resolve on the production host.

legoTest.html:427 | idle.html:1

P1 / S 2. Use one proximity rule for prompts and entry

The nearest portal is assigned regardless of distance; the distance threshold only controls its prompt. The E interaction checks that portal without the same threshold. Share one canInteract predicate between display and action, and consume a just-pressed input so holding E cannot repeat entry.

Done when: E far from posters does nothing; E near a highlighted poster opens that destination once; hidden prompts never imply eligibility.

legoTest.html:1912 | legoTest.html:1963 | legoTest.html:1972

P1 / S 3. Make the poster fallback actually replace textures

A failed image load disposes its texture but never attaches the generated placeholder to the poster material. Returning tex or placeholder cannot handle asynchronous failure because tex already exists. Load with a placeholder installed first, then swap only after successful completion and dispose unused resources.

Done when: Blocking a poster image leaves a readable placeholder and working navigation; successful loading replaces it without retaining unused textures.

legoTest.html:1334 | legoTest.html:1346 | legoTest.html:1355

P2 / M 4. Create a maintainable and accessible gateway

Project metadata, world logic, input and audio share one large file; movement relies on keyboard and mouse, and portals replace the current page. Extract a shared project manifest and focused modules, add an HTML project drawer and touch actions, and preserve optional exploration progress when returning.

Done when: Phone visitors reach all projects; returning preserves optional progress; one manifest update changes the hub and both theaters consistently.

legoTest.html:423 | legoTest.html:492 | legoTest.html:2030

Distinctive next step. Organize curated exhibitions for puzzles, arcade games and engineering experiments. Each poster should offer a making-of card or preview. Keep VIP rewards optional and make the project drawer always available.

Snake Game

snakeGame.html | Review brief 04 of 16

What is worth keeping. A large-map arena combines customization, named opponents, a minimap, and a leaderboard. The optional authoritative WebSocket server and separate snapshot cadence provide a useful foundation for online play. Live name and color previews give the experience personality.

Browser: Local + online checked Local Start displayed a static arena; mouse/Space produced no progression. A separate local WebSocket session joined successfully and moved the player, but steering attempts did not visibly redirect it and no food appeared. Two-client synchronization and reconnect were not tested.

P0 / M 1. Restore movement and authoritative food

Local mode only draws; server input steering is cleared before application; snapshots never populate visible food. Restore a shared simulation, reset steering before collecting input, and render authoritative food coordinates. An isolated server check confirmed that downward input leaves a right-facing snake unchanged.

Done when: Both modes support steering, collection, growth, death, and restart; two clients display identical food.

snakeGame.html:490 | snakeGame.html:139 | snakeGame.html:515 | server.js:155 | server.js:158

P1 / M 2. Bound and validate multiplayer state

Every join creates another snake, and disconnect marks only the latest dead. Enforce one entity per connection and remove expired entities. Validate finite coordinates, names, colors, and message shapes; add payload, connection, and message budgets, heartbeat expiry, origin policy, and backpressure handling.

Done when: Repeated joins cannot increase ownership; disconnect returns state to baseline; invalid or excessive messages do not disrupt other players.

server.js:264 | server.js:279 | server.js:290 | server.js:308

P1 / M 3. Finish connection and recovery states

Online mode defaults to localhost, closure merely clears IDs, and input sends at render cadence. Configure the deployed endpoint and add connecting, retry, disconnected, and match-end states. Cap input frequency and interpolate snapshots so network conditions are visible and manageable for players.

Done when: A disconnect gives clear recovery; deployment needs no URL editing; latency simulation preserves understandable movement.

snakeGame.html:118 | snakeGame.html:181 | snakeGame.html:472 | snakeGame.html:580

P2 / L 4. Complete arena rules before adding spectacle

Server physics collect food and kill on walls but have no snake-body collision stage; AI behavior is basic food seeking. Define consistent body and head collisions, safe spawning, and readable death feedback, then add short rounds that demonstrate distinct rival personalities.

Done when: Scripted wall, body, simultaneous-collision, and spawn cases resolve consistently; each round ends with readable results.

server.js:129 | server.js:154 | server.js:202

Distinctive next step. Position Snake as a compact rival arena with recognizable AI personalities and short challenge rounds. Reliable steering, visible objectives, and fair collisions should precede additional effects or multiplayer expansion.

Blastblock

blastblock.html | Review brief 05 of 16

What is worth keeping. The 8-by-8 puzzle already combines three-piece hands, row and column clears, consecutive-clear combos, all-clear bonuses and persistent records. Gradient pieces and particles provide feedback. Placement and move-exhaustion checks give it a useful functional foundation for a reliable and more accessible, deliberately balanced flagship puzzle game.

Browser: Partial / input limit The board, pieces and Reset loaded. Two browser drags did not place a piece. The input tool may not reproduce native HTML drag-and-drop, so this is not a confirmed user-facing drag failure. The piece tray was partly below the visible fold.

P1 / M 1. Align placement with the visible grid

The grid has 10px padding, but drop coordinates divide distance from its outer edge by 42 without subtracting that padding. This misidentifies cells near their edges. Use measured cell geometry for hit testing and a placement preview, preserving the grabbed square's offset within the piece.

Done when: Verify every cell's corners and center at several viewport sizes and zoom levels; placement previews must match the occupied squares.

blastblock.html:31 | blastblock.html:375

P1 / M 2. Support touch, keyboard and smaller screens

Pieces use native drag-and-drop exclusively, while board and piece cells stay fixed at 40px. The board alone is 354px wide before body padding. Add pointer-based placement, keyboard selection and responsive cell sizing, with clear focus, valid-target and invalid-target feedback.

Done when: Complete separate touch-only and keyboard-only runs; fit the full board at 320px viewport width and preserve accurate placement after resizing.

blastblock.html:33 | blastblock.html:250 | blastblock.html:268

P2 / M 3. Make puzzle rules deterministic and testable

The catalog repeats shapes, hands use random-comparator sorting, and drop payloads are parsed without validation. Extract placement, clears, scoring and seeded hand generation into pure functions. Make any intentional shape weighting explicit and retain the selected piece in application state rather than trusting arbitrary dropped text.

Done when: Cover cross-clears, all-clear bonuses, blocked placements and exhausted moves; identical seeds reproduce hands, and unrelated dropped text causes no exception.

blastblock.html:163 | blastblock.html:296 | blastblock.html:380

P2 / S 4. Retain results and isolate mode records

Game over opens a blocking alert and immediately resets the board. Both Blastblock versions also share the blastblockHighScore cookie despite different board sizes. Add an explicit finished state with results and Replay, cancel pending reset callbacks, and store records under separate mode identifiers.

Done when: Results retain the final board until Replay; each mode preserves its record, and stale callbacks cannot reset a new run.

blastblock.html:208 | blastblock.html:368 | blockblastcopy.html:151

Distinctive next step. Make Blastblock the canonical puzzle project, with seeded daily challenges, optional undo and an end-of-run combo summary. Offer the 9-by-9 prototype as a named mode sharing the same tested engine.

Blastblock Classic 9-by-9 Prototype

blockblastcopy.html | Review brief 06 of 16

What is worth keeping. This larger-board variation demonstrates iteration through a different shape catalog, the same familiar three-piece rhythm, line clears and combo scoring. It is useful as a documented prototype or an explicitly named alternate ruleset. A shared engine could preserve its differences while reducing ongoing duplicated maintenance.

Browser: Partial / input limit The board and hand rendered; a drag did not produce placement. Empty parts of the plus/T silhouettes appeared as colored cells, matching the CSS conflict. A complete scoring run was not validated.

P1 / S 1. Show empty shape squares correctly

Every block cell receives a gradient background image, but the hidden-cell class only makes its background color transparent. Shape zeros therefore retain that image. Remove the background image and other visible decoration for empty cells so the preview communicates the actual placement footprint.

Done when: Inspect every catalog shape: all zero-valued squares must be visually empty, and the visible footprint must match occupied board cells.

blockblastcopy.html:48 | blockblastcopy.html:56 | blockblastcopy.html:181

P1 / M 2. Correct hit testing and device fit

Drop math measures from the outer grid edge without accounting for 10px padding. Fixed board dimensions and a nonwrapping palette constrain narrow screens, and placement only supports drag-and-drop. Share responsive geometry and pointer/keyboard placement with the canonical game instead of maintaining another independent implementation.

Done when: Place pieces accurately at cell edges; complete touch and keyboard sessions, with no clipped board or palette on narrow viewports.

blockblastcopy.html:17 | blockblastcopy.html:34 | blockblastcopy.html:273

P2 / S 3. Declare state and separate records

placedBlocks is assigned and incremented without appearing in the declared state, creating an implicit global in this non-strict script. The high-score cookie also collides with the 8-by-8 version. Declare all state explicitly, adopt strict/module execution, and namespace records by ruleset.

Done when: Run under strict/module rules without undeclared-variable errors; reload both versions and confirm their independent records remain correct after new scores.

blockblastcopy.html:112 | blockblastcopy.html:207 | blockblastcopy.html:151

P2 / M 4. Define its identity and ending flow

This page uses the same title as the newer game, and game over only schedules an alert without an ended state. Present it as Classic 9-by-9 or an archived prototype; explain its rule differences and add a single results view with intentional replay behavior.

Done when: Results appear once and lock placement until Replay; the project entry explains the variant's rules and relationship to the flagship.

blockblastcopy.html:6 | blockblastcopy.html:97 | blockblastcopy.html:266

Distinctive next step. Use the larger board for spatial-planning challenges. Configure board size, shapes and scoring in one engine, and document this version's evolution in the portfolio instead of presenting an unexplained duplicate.

Doodle Jump

doodleJump.html | Review brief 07 of 16

What is worth keeping. The vertical platformer has a substantial progression foundation: springs, breakable platforms, aliens, projectiles, collectible jetpack fuel, achievements and persistent high scores. The jetpack's limited fuel and mid-air refill badge offer useful differentiation. Its next gains should come from consistent simulation, readable controls and intentional progression.

Browser: Short interaction The character bounced and responded to direction taps; a projectile was visible after keyboard input. The score stayed at zero in this short run. No instructions were visible, and a complete climb or post-loss run was not validated.

P1 / M 1. Define gameplay after a loss

endGame shows results, but updates and input continue. The Ghost in the Sky achievement explicitly rewards scoring after game over, suggesting this may be intentional. Freeze normal results, or expose a clearly named optional ghost mode with separate scoring and an explicit transition back to replay.

Done when: Normal-mode score, position and achievements stop changing after loss; any ghost mode is labeled, and Replay restores a clean run.

doodleJump.html:99 | doodleJump.html:288 | doodleJump.html:293 | doodleJump.html:311

P1 / M 2. Decouple simulation from frame rate

Gravity, movement and alien spawning advance per update, while jetpack duration is a 300-frame counter. Different update rates therefore change movement, encounter frequency and fuel duration. Introduce a fixed simulation step or elapsed-time units, cap catch-up work and explicitly pause on visibility changes.

Done when: At 30, 60 and 120Hz, compare jump height, travel, jetpack duration and alien frequency; tab switching must preserve controlled behavior.

doodleJump.html:184 | doodleJump.html:233 | doodleJump.html:236 | doodleJump.html:316

P1 / M 3. Detect platform crossings reliably

Landing checks only whether current feet occupy a platform's thin vertical band, so sufficiently fast falls can skip it. Track previous and current positions to detect downward surface crossings. Also defer removals and recycling until after iteration instead of splicing the platform array during forEach.

Done when: Script several downward velocities and platform crossings; valid landings always bounce, breakable platforms disappear once, and recycling preserves intended spacing.

doodleJump.html:253 | doodleJump.html:257 | doodleJump.html:269

P2 / M 4. Make onboarding and movement accessible

Play begins immediately with a fixed 400-by-600 canvas and side panel, hidden page overflow and no touch controls or visible tutorial. Releasing either horizontal key also cancels movement regardless of the other held key. Add Start/Pause/help, responsive layout, touch controls and key-state-based movement.

Done when: A player can discover every action; mobile play remains visible, simultaneous direction keys behave predictably, and focus loss clears movement.

doodleJump.html:9 | doodleJump.html:23 | doodleJump.html:35 | doodleJump.html:311 | doodleJump.html:328

Distinctive next step. Build an original sky-exploration identity around themed altitude zones, deliberate mission badges and meaningful jetpack decisions. Add a predictable difficulty curve and reachable platform generation before expanding the enemy catalog.

Age of War

ageOfWar.html | Review brief 08 of 16

What is worth keeping. The lane-strategy prototype combines three eras, nine units, ranged and melee combat, defensive turrets, kill bounties and adjustable enemy spawn difficulty. Era definitions already collect balancing data in one place. This is a useful base for demonstrating intentional economy design and clear, readable tactical decisions.

Browser: Played opening battle Started Easy, purchased Clubman and Slingshot units, then bought a Volcano Turret. Units entered combat. Upgrade Base gave no explanation when unaffordable. The letter-based units were difficult to distinguish by role. Match completion was not tested.

P1 / M 1. Run economy and combat on one clock

Gold, movement, melee damage and ranged cooldowns advance per animation frame, but enemy spawns and upgrades use wall-clock intervals. Faster rendering therefore changes the economy relative to the opponent. Convert all rates and timers to simulation time, with consistent pause and bounded catch-up behavior.

Done when: A seeded 60-second run produces equivalent income, movement, damage and spawns at 30, 60 and 120Hz; pause suspends every subsystem.

ageOfWar.html:838 | ageOfWar.html:1007 | ageOfWar.html:1041

P1 / M 2. Control turret fire and projectile workload

Each turret creates a new projectile every update while an enemy is in range. Every projectile then receives individual DOM position writes. Add explicit fire cooldowns, lifetimes and bounded projectile counts or pooling; profile busy battles before deciding whether rendering needs to move to Canvas.

Done when: Configured shots per second stay consistent; crowded matches keep projectile and DOM counts bounded and recover after all targets disappear.

ageOfWar.html:620 | ageOfWar.html:640 | ageOfWar.html:971

P1 / M 3. Complete the match lifecycle

Ending a match shows an alert and stops timers, but there is no replay/menu flow or comprehensive state reset. Turret purchase and removal also lack the ended-state guards used elsewhere. Add a single lifecycle controller that resets entities, economy, eras, audio and scheduled work.

Done when: Win-to-replay and loss-to-menu-to-start both create pristine matches; ended matches reject purchases, and exactly one timer and loop set remains active.

ageOfWar.html:558 | ageOfWar.html:582 | ageOfWar.html:1020 | ageOfWar.html:1032

P2 / M 4. Explain strategic choices and affordability

Unit buttons reveal only name and cost; the upgrade button omits its price, and unaffordable actions silently return. Enemy units and upgrades do not use a tracked economy. Show roles, stats, affordability and upgrade benefits, then explain and balance the opponent's intentionally asymmetric rules.

Done when: New players can identify affordable counters and upgrade costs; documented difficulty profiles produce measurable, intentional differences across representative full matches.

ageOfWar.html:256 | ageOfWar.html:484 | ageOfWar.html:531 | ageOfWar.html:753 | ageOfWar.html:775

Distinctive next step. Develop an original timeline-defense campaign with mutually exclusive era upgrades, unit synergies and battle summaries. Establish fair timing, a responsive battlefield and player-controlled audio before adding more eras or units.

Stick Soccer Showdown

stick/index.html | Review brief 09 of 16

What is worth keeping. The first-to-three match is easy to understand and already includes selectable difficulty, double jumps, ball damping, a speed cap and high-DPI rendering. The small physics sandbox supports quick experimentation. Clear rules, dependable scoring and a robust match lifecycle would substantially improve its presentation for players.

Browser: Short interaction Started a Normal match and tried direction, Space and X inputs while the ball and opponent moved. No goal was scored in the session. The compact controls text disappeared during play; goal/replay edge cases remain source-based findings.

P1 / S 1. Match instructions to actual controls

The menu advertises Space/X as kick controls, but Space actually jumps while X/K kick. This mismatch makes the first match harder to understand. Choose one binding scheme, update the menu and add an in-match legend that explains movement, double jump and kicking consistently.

Done when: Every displayed binding performs its advertised action; a first-time player can discover double jump and kick without inspecting the source.

stick/index.html:30 | stick/index.html:176 | stick/index.html:177

P1 / M 2. Cancel stale goal transitions

A nonfinal goal schedules an unconditional one-second reset and loop restart. Back to Menu does not cancel that timeout, even though the button is available during the goal overlay. Store and cancel pending transitions, and guard callbacks with the active match identity and phase.

Done when: Return to menu immediately after a goal and remain stopped; starting another match must retain clean scores and one loop.

stick/index.html:49 | stick/index.html:152 | stick/index.html:197

P1 / M 3. Score across the visible goal mouth

The drawn goal is 100px tall, but scoring only checks balls whose bottom reaches the last 5px above ground. Shots inside the upper visible goal can therefore fail to score. Define one goal rectangle and crossing rule shared by drawing, wall collision and scoring.

Done when: Low and airborne shots through the opening score once; shots above the crossbar do not, and bounces cannot cancel goals.

stick/index.html:123 | stick/index.html:192 | stick/index.html:216

P2 / M 4. Normalize physics and opponent reaction

Physics advances per frame, but the player's kick cooldown uses milliseconds. The AI applies a kick impulse on every overlapping frame without that cooldown. Use consistent simulation time and explicit AI reaction/cooldown limits, then tune difficulty using measurable parameters rather than only chase acceleration.

Done when: Ball motion remains comparable at 30, 60 and 120Hz; opponent kicks obey documented limits, and each difficulty has behavioral differences.

stick/index.html:84 | stick/index.html:162 | stick/index.html:177 | stick/index.html:185

Distinctive next step. Add local two-player matches, skill-shot practice and a short tournament ladder. Pair these with responsive field sizing and touch controls so the physics game works as a repeatable social experience.

Cheeseburger Empire

idle.html | Review brief 10 of 16

What is worth keeping. An immediately understandable grill action, three business tiers, and collectible production boosts form a compact idle prototype. Its small scope makes it suitable for demonstrating a complete, polished progression loop with clear feedback and dependable saved progress.

Browser: Played + reload check Grilled 50 burgers, bought a Dorm Shop and observed 1 burger per second with the next shop priced at 75. Reload reset ownership and production to zero. The unlimited bulk-sale issue was established from source, not reached in this run.

P0 / M 1. Replace the unlimited bulk-sale loop

Bulk selling subtracts 500 burgers and adds 1,000 without cooldown or a changing requirement. After reaching 500, repeated clicks generate unlimited net gains. Replace the action with limited orders, or separate burger inventory from cash so selling consumes stock and upgrades spend revenue.

Done when: Repeated sale attempts cannot create free inventory; orders respect demand and every transaction conserves its intended balances.

idle.html:138 | idle.html:171

P1 / M 2. Preserve progress and account for elapsed time

All progress initializes from zero, and production adds one second of income per timer invocation without persistence. Add a validated, versioned save and timestamp-based production with capped offline earnings. Reuse a small storage utility with Cosmic while keeping each game's economy independent.

Done when: Reload preserves ownership; delayed callbacks earn the correct elapsed production; damaged saves recover safely.

idle.html:131 | idle.html:182

P1 / S 3. Explain purchases and temporary boosts

Owned business counts stay internal, unaffordable purchases trigger blocking alerts, and the two-minute boost has no countdown. Show ownership, production gain, affordability, and remaining boost time; explain whether another collectible extends an active boost, and replace alerts with inline purchase feedback.

Done when: Players identify the next affordable purchase and boost duration without trial clicks; insufficient funds never interrupt play.

idle.html:122 | idle.html:162 | idle.html:165 | idle.html:209

P1 / S 4. Complete collectible behavior and access

Bonus burgers are clickable divs, and their named floatBurger animation has no matching keyframes. Artwork also depends on an external host. Use labeled buttons, implement the intended motion with a reduced-motion alternative, and bundle licensed artwork so the bonus remains identifiable.

Done when: Keyboard, pointer, and touch can collect bonuses; reduced motion works; essential artwork survives blocked external requests.

idle.html:87 | idle.html:109 | idle.html:189

Distinctive next step. Build a shift-based restaurant game with limited orders, menu decisions, and customer satisfaction. This direction fixes the current sale loophole and clearly differentiates the project from the cosmic incremental game.

Cosmic Cookie Clicker

cookieClicker.html | Review brief 11 of 16

What is worth keeping. Twelve production buildings, configurable upgrades, unlock progression, news, particles, and local saving create a substantial incremental-game foundation. Data arrays make balance changes approachable, while click power linked to production connects active play with the growing economy.

Browser: Played + save check Clicked 20 times, bought one Cursor for 15, saved and reloaded. Ownership persisted. The production display still read 0 per second despite the Cursor's fractional output. Early purchases and visual feedback felt responsive.

P1 / M 1. Protect progress and support returning players

Saves parse without recovery and shallow-merge nested data; saving occurs manually or every 30 seconds. Add schema validation, version migrations, deep default merging, storage-error feedback, and saves at meaningful transitions. Store an update timestamp for capped offline earnings and a return summary.

Done when: Corrupt, older, incomplete, and unavailable saves never block startup or create invalid balances; reload preserves purchases.

cookieClicker.html:527 | cookieClicker.html:532 | cookieClicker.html:536 | cookieClicker.html:545

P1 / M 2. Make purchases reachable on every input

The cookie responds only to mousedown; purchases use clickable divs and upgrade details rely on hover. The fixed-height, overflow-hidden mobile column also needs checking. Use semantic controls, a unified activation path, persistent descriptions, and mobile tabs or a fully scrollable layout.

Done when: Keyboard and touch users can earn and purchase; every panel remains reachable at 360px width and enlarged text.

cookieClicker.html:121 | cookieClicker.html:149 | cookieClicker.html:339 | cookieClicker.html:391 | cookieClicker.html:421

P1 / M 3. Update the economy without rebuilding controls

Every integer-cookie change can empty and rebuild both shops, potentially every animation frame at high production. Create controls once and selectively update prices, ownership, affordability, and unlocks. Refresh passive statistics at a modest cadence while preserving focus and interaction state.

Done when: A high-production save retains focus and reliable purchases without rebuilding the shop each frame.

cookieClicker.html:328 | cookieClicker.html:365 | cookieClicker.html:492

P1 / S 4. Correct rate displays and upgrade eligibility

The number formatter floors 0.1 CPS to zero; a CPS unlock compares lifetime cookies; purchase logic checks funds without checking ownership or eligibility. Separate currency and rate formatting, and share one eligibility function between display and purchase logic, including duplicate protection.

Done when: One cursor shows 0.1 CPS; unlocks use their documented metric; duplicate upgrades cannot apply twice.

cookieClicker.html:197 | cookieClicker.html:237 | cookieClicker.html:375 | cookieClicker.html:514

Distinctive next step. Give Cosmic a strategic identity through planetary delivery contracts, specialization bonuses, and bulk buying. Let players make a meaningful production choice early; keep restaurant operations exclusive to Cheeseburger Empire.

Sharknado Adventure

sharkNado.html | Review brief 12 of 16

What is worth keeping. The illustrated branching story already provides native-button choices and cumulative scoring. A static graph check verified 63 fully reachable nodes, 62 choice edges and 32 endings, all exactly five decisions away. No destination is missing; final scores range from 15 to 80 across complete routes.

Browser: Completed one route Chose emergency kit > search house > follow map > solve puzzle > search for survivors. Reached Hopeful Unity at 75 points after five choices; Restart returned to 50. Some image transitions lagged or appeared empty; all remote assets were not audited.

P1 / M 1. Make scene imagery dependable

The local opening image requests sharkNado.webp, but the tracked file is sharknado.webp; fix this case-sensitive deployment mismatch. The other 62 images depend on remote URLs without fallbacks. Verify asset provenance and availability, package approved images, and provide resilient loading states with descriptive scene alternatives.

Done when: Record each asset's source and permission; failed image loads never block choices, and scenes have descriptive alternatives and consistent sizing.

sharkNado.html:52 | sharkNado.html:63 | sharkNado.html:71 | sharkNado.html:449

P2 / S 2. Reconcile points with story consequences

Several endings contradict their incoming point changes: retreating and running to safety add five points while the prose describes a loss; scouting adds zero while claiming lost points. Audit every edge against its destination, define what score represents, and explain consequences consistently to the player.

Done when: Every route's displayed total matches its choices; each narrative description agrees with the associated point change and documented scoring meaning.

sharkNado.html:242 | sharkNado.html:368 | sharkNado.html:266 | sharkNado.html:392 | sharkNado.html:299 | sharkNado.html:428

P2 / L 3. Let earlier decisions affect later options

The only run state is score; each choice selects a fixed next node without inventory, flags or conditional options. Add a small number of understandable resources, such as supplies and trust, so earlier actions unlock later approaches. Persist progress and collect discovered endings to support purposeful replay.

Done when: At least three later situations respond to prior decisions; progress and collected endings survive reload without changing completed route scores.

sharkNado.html:58 | sharkNado.html:61 | sharkNado.html:474

P2 / M 4. Improve mobile and assistive navigation

The page lacks viewport metadata, and every choice replaces the passage and buttons without deliberate focus management or change announcements. Add a responsive reading layout, move focus predictably to the new passage, expose score changes accessibly and retain clear keyboard focus styling for all choices.

Done when: Finish a route on mobile and with keyboard alone; assistive navigation identifies the new passage, score changes and available choices.

sharkNado.html:3 | sharkNado.html:31 | sharkNado.html:453 | sharkNado.html:470

Distinctive next step. Develop an original illustrated disaster story with recurring characters and intersecting routes. Favor a few memorable resource consequences over a larger binary tree, and retain graph validation as content expands.

Chain Reaction / Color Wars

colorWars.html | Review brief 13 of 16

What is worth keeping. Configurable board sizes, two to eight players, chain reactions, elimination, and current-player colors form a promising shared-device strategy game. Its DOM-based board can support clear labels and keyboard navigation without changing rendering technology.

Browser: Observed startup blocker The controls appeared but no board loaded. Clicking New Game did nothing. Browser logs reported: Identifier p has already been declared. No legal move could be attempted.

P0 / S 1. Remove execution-blocking duplicated code

updateUI redeclares const p and const cssVar in the same scope, preventing the entire script from parsing. After that repair, render still references a loop-scoped orb variable outside its scope. Remove the duplicated fragments and add a startup-plus-first-move smoke check.

Done when: The script parses, the board appears, and the first move advances the turn without an exception.

colorWars.html:168 | colorWars.html:185 | colorWars.html:317 | colorWars.html:326

P1 / S 2. Publish one consistent rule set

Instructions allow an empty cell only on the game's first move, while code allows each player's first move. Critical mass is always four, contradicting the neighbor-based documentation. Choose an intentional rule set and align code, examples, and help; label this implementation local multiplayer.

Done when: Corner, edge, first-turn, elimination, and victory examples agree with gameplay and the written rules.

colorWars.html:74 | colorWars.html:83 | colorWars.html:209 | docs/games/newer-games.md:11

P1 / S 3. Validate board configuration before allocation

Input markup advertises size limits, but JavaScript parses arbitrary values and directly allocates the board. Validate bounded whole numbers in the game model, show inline correction messages, and preserve the last valid configuration before any grid allocation or rendering occurs.

Done when: Negative, fractional, excessive, blank, and pasted dimensions cannot create invalid or oversized boards.

colorWars.html:46 | colorWars.html:131 | colorWars.html:137

P1 / M 4. Make the board responsive and accessible

Fixed-size cells, a horizontal status layout, and clickable divs constrain smaller screens and keyboard users. Add responsive sizing, stacked status, keyboard navigation, ownership symbols, and labeled counts. Animate queued reactions with input locked during resolution, while offering instant reduced-motion playback.

Done when: Phone and keyboard users complete legal moves; grayscale players identify ownership; reaction playback cannot accept overlapping input.

colorWars.html:13 | colorWars.html:29 | colorWars.html:156 | colorWars.html:246

Distinctive next step. Develop a polished pass-and-play strategy game with readable reactions, save/resume, and shareable move replays. Add challenge puzzles after rules stabilize; online rooms should follow a dependable local experience.

NULL - Sensor Puzzle

sensor-puzzle/index.html | Review brief 14 of 16

What is worth keeping. Four minimalist physical puzzles turn stillness, shaking, rotation, and device position into interaction. Persistent completion, listener cleanup, a PWA manifest, and an offline shell already support the concept. NULL is a distinctive candidate for a focused interaction-design case study.

Browser: Desktop only Allowed motion onboarding, opened Be Still and later saw 1/4 solved without physical device interaction. Opened the shake puzzle and used Back. This validates desktop navigation only; sensor samples, phone permissions and actual physical puzzles remain unverified.

P1 / M 1. Detect usable sensors and explain failures

Initialization checks API existence rather than usable samples and requests both motion and orientation permission; denial has no visible explanation. Request only needed capabilities, detect sample arrival, and distinguish denial from unavailable hardware or data. Offer retry and a clearly labeled alternative interaction mode.

Done when: Granted, denied, missing, partial, and silent sensor states each provide useful feedback; real-phone testing covers supported devices.

sensor-puzzle/index.html:348 | sensor-puzzle/index.html:358 | sensor-puzzle/index.html:365

P1 / S 2. Recover progress and provide replay

Startup parses progress without recovery, solving writes without error handling, and completed nodes lose activation. Validate known puzzle IDs within a versioned save, handle unavailable storage, and add replay/reset controls so a damaged save or completed collection does not end access to the experience.

Done when: Malformed or unavailable saves never block puzzles; unknown IDs are discarded; completed puzzles can be replayed intentionally.

sensor-puzzle/index.html:314 | sensor-puzzle/index.html:402 | sensor-puzzle/index.html:610

P1 / M 3. Isolate offline storage and portable installation

Worker activation deletes every other cache on the origin, the precache omits icons, and the manifest uses an origin-root start URL. Restrict cleanup to app-owned cache names, version required assets, and use a deployment-relative launch path with clear update and offline states.

Done when: Unrelated caches survive activation; installed launch works under a repository subpath; cached gameplay and icons work offline.

sensor-puzzle/sw.js:2 | sensor-puzzle/sw.js:13 | sensor-puzzle/manifest.json:5

P1 / M 4. Require fresh samples and cancel delayed work

Stillness continues on a clock after a qualifying sample without checking freshness; cleanup omits delayed shake feedback and home navigation. Reset holds when samples stop or the page hides, track timeouts, bind callbacks to the active puzzle session, and calibrate thresholds using real devices.

Done when: Backgrounding or lost samples cannot complete a hold; leaving a puzzle prevents stale feedback and navigation.

sensor-puzzle/index.html:452 | sensor-puzzle/index.html:476 | sensor-puzzle/index.html:537 | sensor-puzzle/index.html:621

Distinctive next step. Preserve the restrained physical-puzzle identity while adding accessible alternatives, optional hints, replay, and a clear completion moment. A short second chapter can combine gestures after reliable real-device calibration.

Modular 2D Engine - Snake Demo

Game Engine/index.html | Review brief 15 of 16

What is worth keeping. Small modules already separate input, entities, levels, settings and rectangle collision. The Snake example demonstrates entity extension and collision callbacks without a build step. The project plan correctly distinguishes future sprite, physics, audio and camera systems, making this a useful foundation for an educational engine.

Browser: Played restart/turn The grid demo started immediately and reached Game Over. Space restarted it, and a Down input changed the movement direction. The controls are not explained before play. Food collection and a full-board win were not tested.

P1 / S 1. Correct the departing-tail collision rule

Self-collision checks every current segment before removing the tail. On a non-growing move, entering the tail's departing cell therefore causes an incorrect death. Determine the next occupied body before collision checks, and retain the tail in that check only when growth prevents its departure.

Done when: Entering a vacated tail cell survives; entering a retained body cell or a tail retained by growth ends the run.

Game Engine/src/main.js:69 | Game Engine/src/main.js:80

P1 / S 2. Preserve elapsed movement time

The movement timer resets to zero after each step, discarding interval remainder and making speed depend on frame cadence. Use a fixed-step accumulator that subtracts elapsed intervals, with a bounded catch-up policy. Add explicit pause and resume behavior for hidden tabs.

Done when: Equivalent elapsed time yields equivalent steps at thirty, sixty and one-hundred-twenty frames per second; resuming never causes uncontrolled catch-up.

Game Engine/src/main.js:45 | Game Engine/src/main.js:215

P1 / M 3. Turn the bare canvas into a usable demo

The page supplies a fixed-size canvas, keyboard movement and no introductory interface. Add responsive display scaling, visible start, pause and restart controls, touch directions and a return link. Clear held input on blur and pointer cancellation, and manage event listener cleanup explicitly.

Done when: A narrow phone viewport shows the entire board and controls; blur clears input; repeated initialization produces no duplicate actions.

Game Engine/index.html:6 | Game Engine/src/main.js:178 | Game Engine/src/input.js:6

P2 / M 4. Demonstrate a reusable engine contract

Game-specific Snake classes occupy the engine entry file; restart directly replaces the level array. Collider accepts a type argument while implementing rectangles only. Separate examples from engine modules, expose clear lifecycle methods, and either implement or explicitly reject unsupported collision types before expanding features.

Done when: Two examples share unchanged engine modules; object removal and teardown work through documented methods; unsupported collider types fail clearly.

Game Engine/src/main.js:17 | Game Engine/src/main.js:277 | Game Engine/src/collider.js:2

Distinctive next step. Create an engine laboratory with hitbox overlays, pause and single-step controls, adjustable tick rate and an entity inspector. Two concise examples using identical modules would make its engineering value tangible.

God Game - Village Sandbox

God game/index.html | Review brief 16 of 16

What is worth keeping. The prototype establishes a Babylon.js village with an orbit camera, procedural scenery and a selection-ring system. Character-loading and click-to-move code provide the basis for direct unit control. Its compact implementation makes it practical to shape one complete, understandable simulation loop before introducing larger systems.

Browser: Observed asset blocker The camera and tree/hut scenery rendered and responded to dragging. Character loads repeatedly failed with assets/dude/dude.babylon not found. No units could be selected. The grass texture displayed a visible watermark; replace it with a documented, suitable asset.

P0 / S 1. Repair character asset resolution

Both loaders request assets/dude/dude.babylon, while the committed model sits directly inside God game. Correct the path and package associated textures together. Add visible loading and error states, a retry option, and readiness checks so commands cannot target units before loading completes.

Done when: All intended units and textures load from deployed paths; failed loads show a recovery message; premature clicks cause no errors.

God game/main.js:51 | God game/main.js:75 | God game/dude.babylon:1

P1 / M 2. Give every unit a stable movement order

Movement updates only the selected character, uses fixed per-frame interpolation, and accepts any picked surface as a destination. Store orders per unit, update movement using elapsed time, restrict commands to walkable ground, and add obstacle-aware routes. Selection should inspect a unit without interrupting its order.

Done when: Several units complete independent orders at consistent speeds across frame rates, remain grounded, and route around huts and trees.

God game/main.js:103 | God game/main.js:125 | God game/main.js:139

P1 / L 3. Build one complete god-game scenario

Current simulation consists of scenery, selection and movement, with no resource or objective system. Build a five-minute scenario: gather wood, repair a shrine, then use one rain blessing to protect the village. Add understandable feedback, a completion state and restart before expanding the world.

Done when: A new player can identify the objective, command resource work, use the blessing, understand the outcome and restart.

God game/main.js:125 | God game/main.js:160

P2 / M 4. Add onboarding and reproducible dependencies

The page uses a placeholder title, omits viewport metadata and instructions, and imports an unversioned engine plus remote textures. Name the experience, explain camera and command controls, show selection feedback, pin the tested dependency, and document asset provenance and fallback behavior.

Done when: Portrait and landscape layouts remain usable; controls are discoverable; selected-unit status is visible; dependency and texture failures are handled.

God game/index.html:5 | God game/index.html:7 | God game/main.js:20 | God game/main.js:43

Distinctive next step. Develop Caretaker Grove: a village where divine interventions have visible tradeoffs. Rain helps crops but threatens low ground; trees provide wood and stabilize soil. Start with one clearly explained interaction.

A consistent portfolio experience

The homepage, cinema screens and Lego posters should describe the same collection. A shared project manifest removes repeated link maintenance while allowing each surface to keep its own personality.

The visitor journey

Step	What the visitor needs	Recommended behavior
Discover	What is this, and can I play it?	Visible title, short premise, readiness/device badge and thumbnail.
Choose	Why is this project interesting?	Play, source and case-study actions; controls and approximate session length.
Play	How do I start and recover?	Loading state, start, help, pause, restart, mute and a return link.
Understand	What did Kai build and learn?	A concise explanation of one challenge, decision, tradeoff and result.
Return	What should I try next?	A related project suggestion and a dependable route back to the catalog.

One record per project

Recommended fields: stable ID, title, description, route, preview image, category, release status, technologies, supported input/device types, controls, source URL and case-study summary. Validate routes and thumbnail files against the tracked repository.

A minimum presentation standard

Use a consistent top-level shell while preserving each game's art direction. Include visible keyboard focus, readable contrast, semantic controls, touch alternatives where appropriate, reduced-motion behavior and meaningful image alternatives. Use plain status labels such as Ready, Experimental and Desktop only.

First case study template Problem and intended player -> what Kai implemented -> one hard technical decision -> screenshot or playable result -> what was tested -> next improvement. Keep claims specific and avoid invented player counts or performance gains.

Source basis: index.html:257; legoTest.html:423. Structural accessibility guidance: W3C WAI Page Structure Tutorial.

A small, dependable engineering foundation

Preserve the easy-to-run character of the repository. A framework rewrite is not required. Extract shared logic only when it eliminates a concrete repeated defect or supports a meaningful test.

One simulation clock

Doodle Jump, Age of War and Stick Soccer mix per-frame motion with wall-clock timers. Use elapsed time or fixed simulation steps, with a bounded catch-up policy and pause-on-background behavior. The 2D engine already accepts delta time but drops interval remainders.

Small state and lifecycle contracts

Define loading, ready, playing, paused and ended states. Own and cancel timers, listeners, animation loops and scene resources. Restart must construct a clean session; leaving a screen must stop its delayed work.

A reusable save utility

Use a versioned format, guarded storage access, schema validation, migration and a last-updated timestamp where offline earnings are intentional. Save on meaningful transitions; explain recovery if stored data is corrupt. Preserve distinct records for different rulesets.

Tests that protect behavior

Add syntax and local-link checks first. Then test economy transactions, movement, collision, scene transitions and save migration. Convert existing screenshot exploration into assertions for real user journeys; direct scene jumps do not prove that doors or controls work.

Controlled deployment and assets

Pin tested runtime dependencies, document the static site and WebSocket service separately, and stage on a case-sensitive host. Package runtime assets intentionally; keep source archives and unused model packs outside the deployment artifact.

Observable online failure

Snake needs message schemas, finite/ranged coordinates, connection cleanup, input/snapshot budgets and readable reconnect states. Measure snapshot size and server update time before selecting player limits; no safe capacity has been established.

Evidence: package.json:1; playwright-test.js:47; playwright-explore.js:44. Browser timing guidance: MDN requestAnimationFrame. Transport guidance: MDN WebSocket.

A practical release roadmap

Treat this as a sequence of release gates, not a promise to implement every recommendation in a month. The first release should deliberately include fewer, stronger projects. Estimates are focused developer time; content, art, device testing and review may extend the calendar.

Gate 1 / Restore trust

Target: first 3-6 developer days

Fix Chain Reaction startup, Snake local simulation/steering/food, God game model paths, Cheeseburger sale accounting and exact-case links. Add parse and link checks so these defects cannot silently return.

Gate Exit: selected public games start, their central loop works, and no known P0 issue remains in the featured lineup.

Gate 2 / Make every visit understandable

Target: next 4-7 developer days

Build the shared project catalog, visible anchor cards, clear controls/return links, loading/error states and consistent results/restart behavior. Pick the featured arcade game; label remaining prototypes honestly.

Gate Exit: a new visitor can find, launch, understand and leave every featured project using its supported input.

Gate 3 / Polish one complete showcase

Target: next 8-12 developer days

Refine one cinema route and one arcade game. Fix timing and input issues in that selected game, write two case studies, and add a direct project drawer. Validate NULL on actual phones if it will be featured.

Gate Exit: complete end-to-end sessions pass the release checklist, screenshots match the shipped version, and case studies substantiate specific claims.

Gate 4 / Broaden deliberately

Target: plan after the first release

Extend the shared quality standard to remaining games, then pursue the distinctive mechanics in each brief. Add multiplayer load testing before public Snake promotion; schedule new content only after retention and playtest evidence exist.

Gate Exit: each new feature has an observable user benefit and a small, relevant acceptance check.

The first ten work items

These first work items are intentionally concrete. The remaining brief actions form the longer backlog. Keep each issue small enough to review and close with its acceptance evidence.

ID	Work item	Acceptance evidence	Effort
01	Restore Chain Reaction startup	Remove duplicate declarations and out-of-scope render reference; board and first turn work.	S
02	Repair Snake simulation contract	Working local loop, retained server steering and authoritative food render; exercise start-to-restart.	M
03	Repair model and route paths	Load God game characters; fix both Idle.html references; exact-case route scan passes.	S
04	Correct burger transactions	Make sales consume real inventory or limited orders; repeated clicks cannot create free profit.	S-M
05	Publish a canonical catalog	Include all intended projects with status, controls and devices; all links and preview assets validate.	M
06	Make the hub operable	Semantic cards, visible titles, labels, focus states, result count and no-results recovery.	M
07	Finish one arcade lifecycle	Choose Blastblock or Age of War; implement start/help/pause/results/replay and return-to-hub.	M
08	Fix that game's core timing/input	Accurate block placement and alternative input, or a single strategy simulation clock and turret cooldown.	M
09	Offer a direct cinema visitor path	Project browser and camera fallback; reach a game without tickets or pointer lock.	M
10	Ship evidence with the release	Meaningful smoke assertions, two case studies, current screenshots and a concise run/deploy README.	M

Dependencies: 01-04 establish basic trust; 05 enables 06 and 09; 07-08 belong to the same selected arcade project; 10 validates the release. The original source can remain modular vanilla JavaScript.

Definition of ready

A project is ready to feature when these checks pass for its declared devices and modes. The numeric targets below are proposed acceptance targets, not measurements collected during this audit.

Launch and navigation

Every public URL and thumbnail resolves on a case-sensitive preview. No uncaught startup exception. A visitor can launch a game from both the catalog and its supported immersive route, then return.

A complete game session

Test start -> core action -> win/loss or completion -> results -> replay. Include pause/resume, focus loss, and leaving during an animation or delayed transition. One active loop remains after each restart.

Input and inclusive access

Keyboard-only menus work, focus is visible, instructions match actions and non-color cues identify players. Test touch when claimed. Check 360px layouts, enlarged text and reduced motion; do not claim a formal accessibility conformance result without a full audit.

Timing and resource stability

At simulated 30/60/120Hz, representative movement, duration and economy remain within an agreed tolerance (suggested 5%). Run 20 scene round trips and a 10-minute game session; no sustained growth in owned resources after warm-up.

Saved progress

Reload preserves supported progress. Older, malformed, missing and unavailable storage recover predictably. Offline earnings follow explicit rules, and different game modes cannot overwrite one another's records.

Multiplayer

Use two real clients for join, movement, collection, death and reconnect. Repeated joins leave one player per connection. Malformed input is rejected; disconnected clients are removed. Set capacity targets only after measuring bandwidth, frame time and server load.

Mobile sensors and PWA

Test real iOS and Android hardware with permission granted/denied, absent samples, backgrounding and interruption. Install from the deployment subpath; verify offline assets and that unrelated origin caches survive updates.

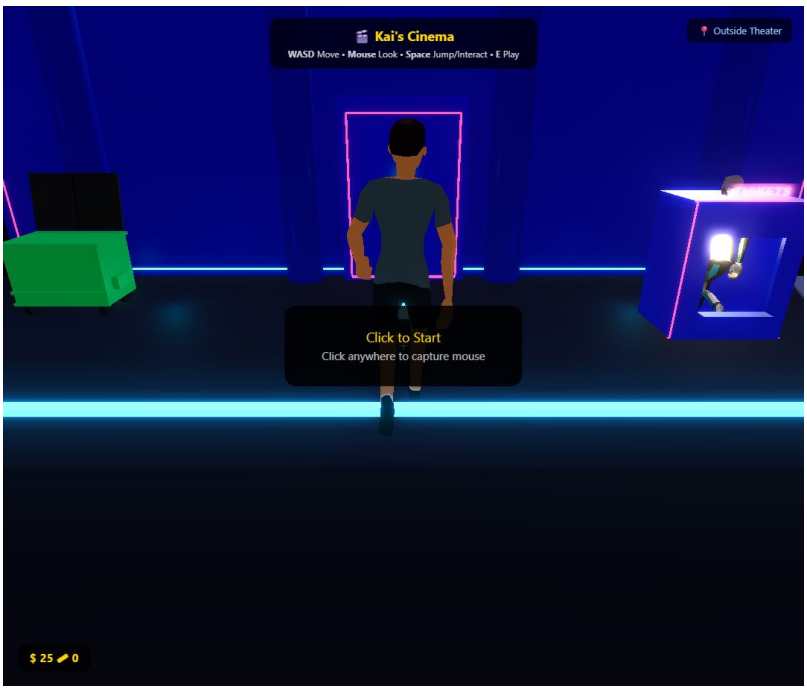
Presentation and learning evidence

Screenshots and docs describe the shipped revision. Featured case studies identify Kai's role, one technical decision, validation and a next step. Asset sources and license records are traceable.

Browser evidence / experience

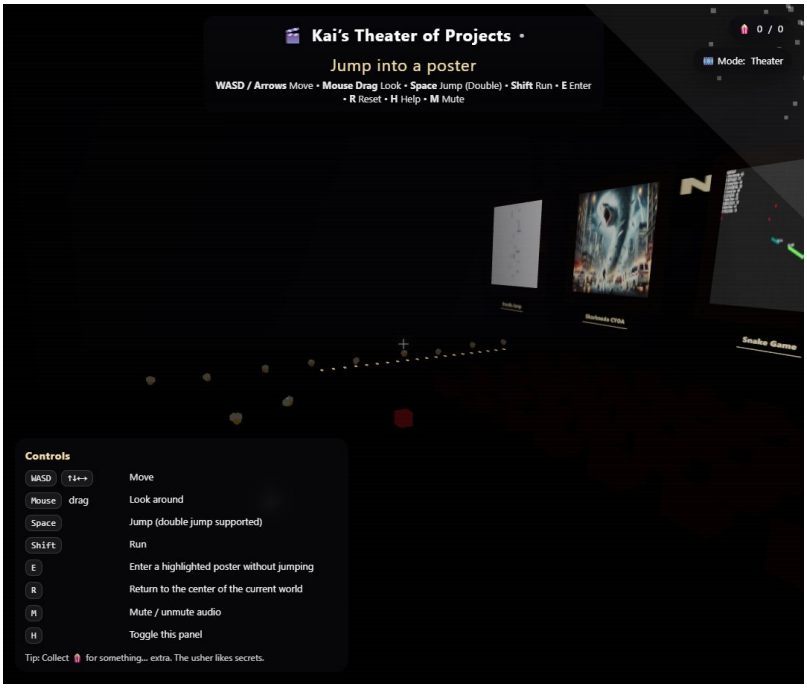
Screenshots were captured during this review from the verified local source. They show actual UI states, not redesigned mockups or old repository screenshots.

Kai's Cinema



The exterior rendered with animated characters and a strong neon identity. The start overlay persisted after mouse-capture failure in this browser. This image is not evidence that interior navigation passed.

Theater of Projects

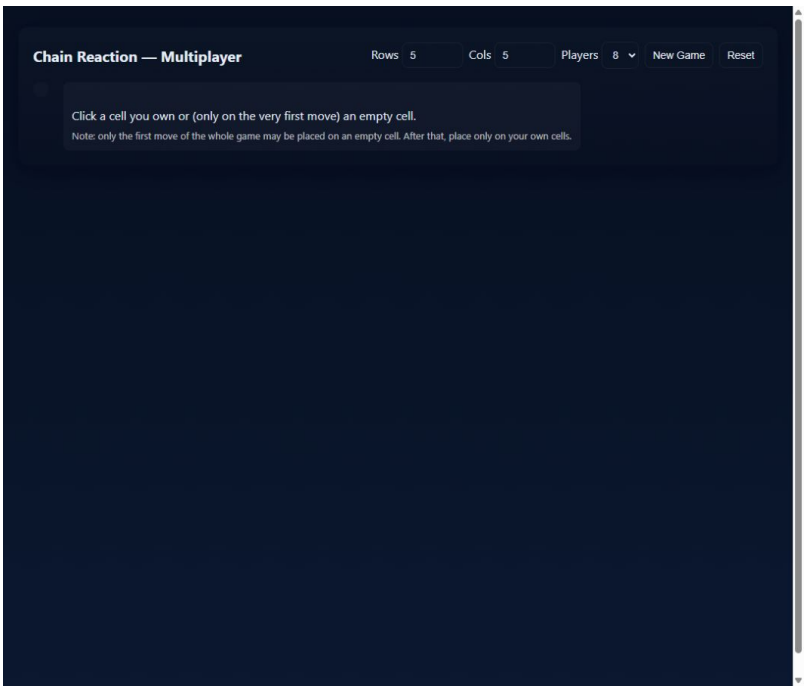


Camera dragging changed the view and the help panel responded. The scene was difficult to read at its current brightness. Treat lighting changes as an aesthetic recommendation, not a performance finding.

Browser evidence / reliability

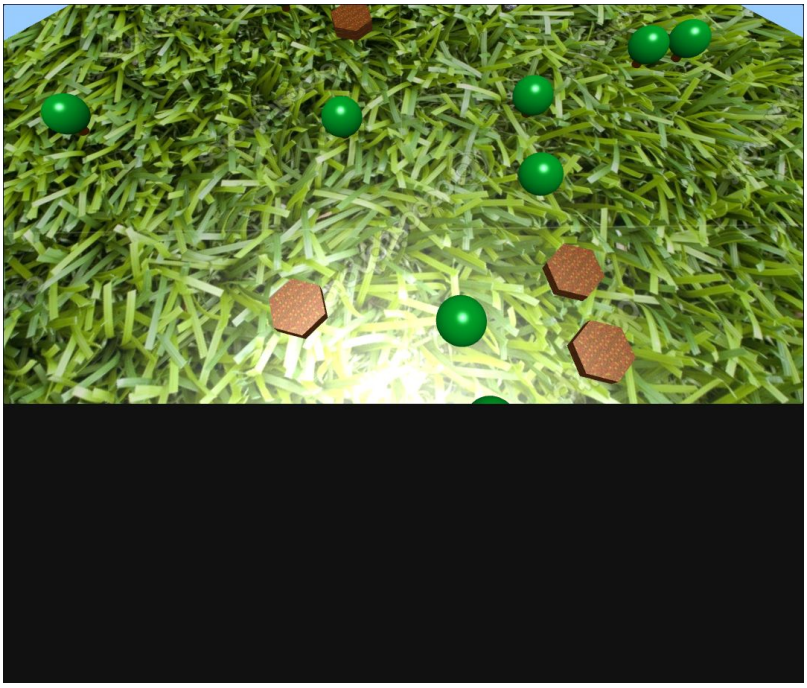
Screenshots were captured during this review from the verified local source. They show actual UI states, not redesigned mockups or old repository screenshots.

Chain Reaction



New Game left the board absent. The browser reported the duplicate p declaration. This is a reproduced startup blocker with a matching source explanation.

God game

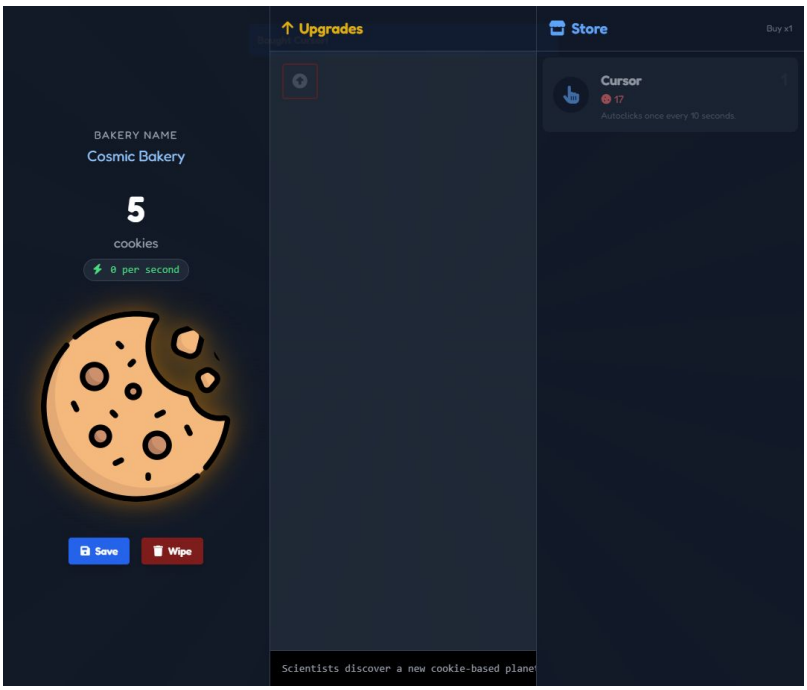


The terrain and trees render, but character model requests fail. The visible grass watermark also weakens presentation; substitute an asset with documented provenance.

Browser evidence / progression

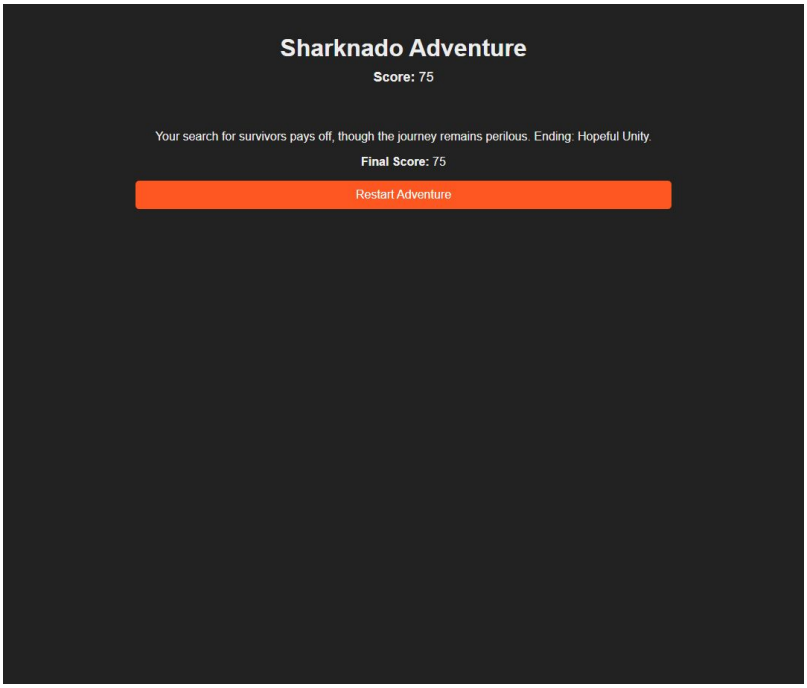
Screenshots were captured during this review from the verified local source. They show actual UI states, not redesigned mockups or old repository screenshots.

Cosmic Cookie Clicker



A Cursor purchase succeeded and survived manual save/reload. Its fractional production was displayed as 0 per second, demonstrating an actual feedback defect.

Sharknado Adventure



A five-choice route reached Hopeful Unity with 75 points. Restart worked. This validates one complete branch; the separate static graph check covered all destinations.

Evidence, provenance and limits

Repository evidence

hasturofyellow/kai-portfolio
Exact reviewed commit: 75a9e07
GitHub metadata and the remote main ref agreed with the clean local tracked source. Individual findings link to permanent file/line locations in that commit. New files created for this report are outside the reviewed snapshot.

What was checked

All sixteen application entry points were source-reviewed and opened in the browser. Short interactions covered the hub, both theaters, arcade games, idle games, the story and sensor onboarding. One story branch was completed; one client joined a local Snake WebSocket server bound to loopback. Additional isolated Node checks confirmed the Chain Reaction parse error, Snake steering/repeated-join defects, and the 2D Snake tail/timer defects. A static story-graph check found 63 reachable nodes, 32 endings, five choices per ending and no missing destinations.

What remains unverified

This is not a deployed-site certification, exhaustive game balance review, full security assessment, formal accessibility audit, mobile hardware test or multiplayer capacity benchmark. Pointer lock failed in the in-app browser; native HTML drag-and-drop could not be completed reliably. Those limits are not generalized into claims that all user browsers fail. No real-device sensor behavior or sustained high-refresh gameplay was measured.

Asset and documentation provenance

The character pack already includes Quaternius / CC0 provenance: Animated Men Characters - Feb 2019/License.txt:2. Preserve it. Build an asset register for the remaining images, models, animations and audio; a missing record is a documentation gap, not proof of improper use. Separate unused source archives from runtime deployment assets.

The cinema roadmap predates existing bloom and NPC work; compare 3D Game Engine/movieTheaterProject.md:202 with 3D Game Engine/src/game/theaterGame.js:86. The tracked worktree gitlink at 3D Game Engine/.claude/worktrees/lucid-booth also deserves repository housekeeping; verify its purpose before removing it.

Technical reference material

MDN: requestAnimationFrame and frame timing

MDN: WebSocket interface and backpressure caveat

W3C WAI: Page Structure Tutorial

External references accessed 28 September 2026. Recommendations and effort estimates are reviewer judgments based on the stated source and testing scope.